

Executable engineering arguments with EAL/2: Typed evidence, defeasible assessment and an API load-test pilot

Review manuscript for the Journal of Systems and Software
25 September 2026

Abstract

Engineering decisions made with language models require a traceable relation between a claim, the observations used to assess it and the rule used to interpret those observations. EAL/2 represents these relations in a domain-specific language and evaluates them through a Python host. The host binds typed propositions to scoped, dated evidence, invokes versioned reasoning methods and computes grounded acceptance in the presence of objections. We describe the implemented semantics, distinguish executable checks from authored explanations, and demonstrate an API performance argument. A reanalysis of an archived development experiment preserves all 1,200 assignments across 40 controlled HTTP cases, six model snapshots and five prompting/tool conditions. Of these assignments, 628 completed, 131 failed and 441 were not attempted after model-level stopping. GPT-4.1 full achieved 40/40 correct assignments with EAL/MCP and 36/40 with a JSON prompt; the corresponding mini results were 32/40 and 21/40. For the full model, all four JSON discrepancies concerned the required set of failed checks, while its decision statuses were correct. Different tool access, purposive case selection and incomplete execution restrict these comparisons to the recorded systems and cases. The contributions are an inspectable implementation, a reproducible account of the pilot, and evidence locating failures in both tool interaction and communication of checked results.

Keywords: engineering argumentation; domain-specific languages; evidence provenance; grounded semantics; language models; software performance assessment

1 Introduction

An engineering claim can be syntactically well formed and numerically plausible while referring to the wrong deployment, an expired observation or an untested assumption. A language model asked to assess that claim must select evidence for the specified deployment and interval, apply the intended rule and communicate the result without changing its scope. Producing a structured answer addresses only part of this task. The relation between the answer and the underlying observations needs an executable definition.

EAL/2, the Engineering Argument Language, makes that relation explicit. A source document names claims, evidence, assumptions, reasoning methods, supporting arguments and objections. A host validates the document, obtains observations through configured collectors, applies computational methods and returns an assessment with diagnostics. Authors can express a proposition using a typed quantity, unit, subject, scope, interval, query and result condition. Those fields constrain which observation may support the proposition. Free-text descriptions remain available for explanation, with an explicit indication that their meaning has not been verified.

The engineering contribution is the combination of these checks in a small executable system. It is useful when an assessment must be repeated after evidence changes or when a reviewer needs

to distinguish a failed requirement from evidence that cannot establish whether the requirement holds. Existing work supplies argumentation semantics, structured assurance representations and model-to-solver techniques. EAL/2 adopts parts of that foundation and defines an operational profile for dated engineering observations and a tool-accessible assessment host.

Two forms of evidence support the account. Source inspection and automated tests establish behaviour of package version 2.5.0. Separately, a supplied archive records an earlier, five-condition development experiment on API load-test assessment. The archive contains actual model calls and HTTP observations, together with failed and unattempted assignments. Its scorer and source snapshot are preserved here. Keeping these versions separate prevents a later implementation change from being credited with an earlier outcome.

The paper addresses three questions. First, which relations between a proposition and an observation does the implementation check, and where does its assurance end? Second, what outcomes did the historical EAL/MCP and comparator systems produce on the assigned cases? Third, which recorded failures identify concrete changes to assessment interfaces or experimental controls? The pilot supports descriptive answers to the latter questions. It does not estimate a general advantage of a notation, establish a model-size effect or measure performance in production services.

2 Related work and design position

Dung’s abstract argumentation framework represents arguments and attacks and defines acceptable sets through their defence relations [1]. Its grounded extension provides a conservative basis for acceptance when attacks remain unresolved. EAL/2 implements an explicit attack-only solver and also a composed evaluator in which argument nodes depend on premise claims and claim nodes have alternative derivations. The composed evaluator therefore requires rules for both support and attack; attaching a familiar argumentation label to an arbitrary dependency graph would be insufficient.

ASPIC+ constructs arguments using strict and defeasible rules and supports attacks and preferences [4]. EAL/2 has named arguments, objections and computational reasoning methods, but the implemented profile does not claim the full ASPIC+ semantics. Its local usability checks and acceptance equations are stated in Section 4. This restricted account makes the implementation’s commitments testable without implying that every property of a broader structured-argumentation formalism transfers to it.

The Structured Assurance Case Metamodel defines representations for assurance claims, arguments and evidence and their interchange [5]. EAL/2 addresses a narrower implementation problem: repeatedly evaluating designated evidence and argument dependencies through executable methods. The comparison concerns the scope of this implementation; assessing execution in other assurance-case tools would require examining those tools individually. A representation of a supporting relation and an implementation that evaluates a particular relation provide different kinds of evidence for a software decision.

Program-aided language models delegate computations in generated programs to an interpreter [2]. Logic-LM translates problems into formal representations, invokes symbolic solvers and interprets the results [6]. These systems establish the relevance of external computation to model-assisted reasoning. EAL/2 similarly delegates defined computations, while adding evidence identity, freshness and objection handling. A solver’s result remains conditional on the supplied formalisation and observations; it does not establish that an informal engineering problem has been represented correctly.

Grammar-constrained decoding studies whether generated outputs conform to a formal grammar and how that affects logical parsing [8]. The pilot reported here supplied EAL source to the model and used a common text-based operation adapter. It consequently measures neither EAL authoring ability nor grammar-constrained generation. Its JSON condition supplied a

semantically corresponding argument as prompt text; it did not invoke a provider’s structured-output feature.

The implementation follows conventional language-engineering separation between parsing, intermediate representation, name resolution and evaluation [7]. This separation permits syntax and reference errors to be reported before evidence is assessed. The host exposes tools through the Model Context Protocol (MCP) [3]. MCP supplies the interaction format; the EAL implementation supplies the meanings of validation, collection and assessment. These protocol and semantic responsibilities must remain distinguishable when interpreting a successful tool call.

3 Language and implementation

3.1 Source records and assessment responsibilities

EAL/2 source declares an environment, tools, evidence, assumptions, reasoning records, claims, arguments and objections. Patterns can generate repeated argument structures. An argument refers to a conclusion, a reasoning record and any evidence, assumptions or premise claims it needs. An objection is evaluated as an argument that attacks a designated target. Several arguments may independently offer derivations of the same claim, while the premises within a particular derivation must all hold.

The Python implementation parses the source with an ANTLR grammar and constructs a resolved intermediate representation. Validation checks identifiers, reference kinds, required fields, method-specific inputs and the acyclicity of ordinary premise dependencies. Resource limits bound processing: source input is limited to 1 MiB, token count to 100,000, records to 4,096, premise nesting to 128 and pattern applications to 1,000. The composed assessment graph admits at most 4,096 argument nodes, 4,096 claim nodes and 131,072 relationships. These bounds specify the admitted inputs to this implementation.

The package requires Python 3.11 or later. The inspected version uses ANTLR runtime 4.13.2 and MCP SDK 1.30.0, with an MCP stdio profile dated 2025-11-25. Package version, language revision and protocol profile identify different artefacts. Reproducing an assessment also requires the source document, method identifiers, collected records and assessment time. The software and data availability statement specifies the source snapshots used for the software description and historical results.

3.2 Evidence collection, identity and time

The host executes collectors configured by its operator. Source documents name tools and inputs; they do not supply an arbitrary executable command. Collector outcomes, including errors, are recorded in SQLite. A record can contain a collection time, content digest, metadata and a structured value used by a reasoning method. Its digest checks content consistency against the recorded value. It cannot establish that a sensor was calibrated, that a remote service was honest or that the observation represents the intended operational population.

Freshness is evaluated against an explicit assessment time. If t is that time, t_c the recorded collection time and a the permitted age, the condition is

$$0 \leq t - t_c \leq a. \tag{1}$$

Preserving the original collection time when importing evidence prevents ingestion from making an old observation fresh. Assumption validity uses a declared interval with an inclusive start and an exclusive end. These temporal checks establish eligibility under the declared rules. They cannot establish that the measured system remained unchanged throughout the interval.

Declared proposition		Supplied observation
$s = \text{pump-A}$	$s = s'$	$s' = \text{pump-A}$
$q = \text{pressure}, \quad u = \text{kPa}$	$q = q'$	$q' = \text{pressure}, \quad u' = \text{Pa}$
$\sigma = \text{trial-v1}$	$\sigma = \sigma'$	$\sigma' = \text{trial-v1}$
$I = [t_0, t_1)$	$I \subseteq I'$	$I' = [t_0, t_1)$
$Q : \text{assignment}=\text{randomised}$	$Q = v _Q$	$v : \text{assignment}=\text{randomised}$
Result conversion: $7500 \text{ Pa} = 7.5 \text{ kPa} \geq 5 \text{ kPa}$		Method: <code>causal/1</code>

Figure 1: Typed correspondence for a constructed `causal/1` example. The two columns denote proposition fields and supplied observation fields; connecting relations specify the checks. The observation interval must contain the proposition interval, and $v|_Q$ denotes the fields of the supplied value selected by the query. Compatible units permit the illustrated conversion before the result condition is checked. The values are stipulated, and the declared randomised assignment is not independently established by this correspondence.

3.3 Typed propositions and computational methods

A typed proposition identifies a subject s , quantity q , unit u , scope σ , interval I , query Q and result predicate R . A designated computational evidence value carries an `EAL/typed-input/1` envelope. The host checks that its method, subject, quantity and scope match the proposition, that its unit is compatible, that its observation interval contains I , and that the query agrees under canonical comparison. The method then computes a result for evaluation against R .

Figure 1 illustrates these correspondences with stipulated pressure observations. The unit conversion allows a result expressed in pascals to be compared with a condition in kilopascals. Agreement of the metadata is necessary for this binding, but matching an authored scope string does not independently verify experimental assignment. The figure illustrates a computational interface using stipulated values.

Eight built-in method families are versioned separately from their display names. The default `structured/1` method checks the availability of declared dependencies while retaining an authored rationale. It does not turn that rationale into a formal entailment. `deductive/1` evaluates bounded propositional inputs, with at most 12 atoms, and checks premise consistency. `inductive/1` uses a Wilson interval; `abductive/1` uses declared priors and likelihoods; and `causal/1` evaluates a contrast between independent groups. `counterfactual/1` operates on a bounded affine structural causal model with fixed exogenous inputs. `analogical/1` compares specified features, and `temporal/1` checks sampled temporal coverage. Each method’s result is conditional on its input schema and assumptions. In particular, sampled temporal coverage does not prove continuous-time behaviour, and a group contrast alone does not identify an intervention effect.

A registered custom method can extend the computations. Resource-limited worker execution bounds some failures, but it does not make custom code a security sandbox or prove its purity. Method registration and collector configuration remain trusted host responsibilities. The assessment includes `prose_verified: false` to identify the separate status of explanatory prose. A reader can inspect what was computed without treating the surrounding natural-language interpretation as mechanically proved.

4 Defeasible assessment and implementation evidence

4.1 Acceptance of arguments and claims

Let N be the finite set of argument and objection nodes and C the finite set of claims. For $n \in N$, let $U(n)$ denote the result of its local usability checks, $P(n) \subseteq C$ its required premise claims and $T(n) \subseteq N$ its attackers. For $c \in C$, let $D(c) \subseteq N$ contain its derivations. Labels range over accepted, rejected and undecided. With A and R denoting the first two labels, the evaluator applies the following rules:

$$n : A \quad \text{if } U(n) \wedge \bigwedge_{p \in P(n)} p : A \wedge \bigwedge_{b \in T(n)} b : R, \quad (2)$$

$$n : R \quad \text{if } \neg U(n) \vee \bigvee_{p \in P(n)} p : R \vee \bigvee_{b \in T(n)} b : A, \quad (3)$$

$$c : A \quad \text{if } \bigvee_{d \in D(c)} d : A, \quad (4)$$

$$c : R \quad \text{if } \bigwedge_{d \in D(c)} d : R. \quad (5)$$

Empty conjunctions are true and empty disjunctions false. Thus a claim with no derivation is rejected by the graph evaluator. This graph label means that no accepted derivation is available; it does not establish the factual negation of the engineering claim.

Evaluation starts with every label undecided and repeatedly applies the rules until no label changes. Settled labels are retained. On a finite graph, this monotone accumulation of information terminates because each node and claim can become settled at most once. This argument concerns termination of the label computation after local checks have completed. Collector execution and custom methods require their own runtime controls.

The rules make conjunctive premises and alternative derivations explicit. An accepted premise alone cannot compensate for another rejected premise of the same argument. An accepted alternative derivation can support a claim whose other derivation has been rejected. Reusing an evidence record across derivations does not create independent observations or multiply the statistical strength of that evidence. Attack cycles may remain undecided even though ordinary premise dependencies are acyclic.

The public engineering statuses include `supported`, `contested`, `unsupported` and `out_of_scope`. They are interpretations of an assessment in its declared environment. Evidence availability, graph acceptance and the truth of an external proposition are separate quantities. An application that needs an `unavailable` decision category must specify its mapping from diagnostics and evidence checks; Section 6 describes the historical experiment's mapping.

4.2 What the tests establish

The repository checks the attack-only solver against independent complete-extension enumeration for all 531 directed graphs with zero to three nodes, including self-attacks. The composed evaluator is compared with an independent Dung translation over 1,024 two-node, one-claim combinations, 200 seeded larger cases and 512 three-node attack-only cases. Additional tests exercise propagation through a chain of 4,096 argument nodes and 4,096 claims across 8,192 rounds. These finite comparisons check the implemented rules on the enumerated and generated inputs; they are not a mechanised proof covering every graph within the resource limits.

For the source snapshot described in this paper, the full test suite completed with 846 passing tests. It includes parser, validation, evidence, reasoning, command-line and MCP interaction tests. Agreement between an implementation and a translated reference could still reflect a shared modelling error, so the equations, independent enumerations and executable tests are supplied together. Test success establishes the checked software behaviour. It cannot establish

that a deployed collector observes the intended system or that a language model will communicate an assessment faithfully.

5 Worked API performance argument

The repository’s worked example concerns a named API service, build, load-test run and concurrency level. A collector validates the report and computes request count, nearest-rank sample p95 latency and the proportion of failed requests. For n ordered latency values $x_{(1)}, \dots, x_{(n)}$, the sample quantile is

$$\hat{q}_{.95} = x_{(\lceil .95n \rceil)}, \quad \hat{e} = \frac{\#\{\text{failed requests}\}}{n}. \quad (6)$$

The declared acceptance conditions are $n \geq 100$, $\hat{q}_{.95} \leq 200$ ms and $\hat{e} \leq 0.01$. Report identity, completeness, freshness and internal consistency must also satisfy their checks. These thresholds specify a decision for a finite report; they do not provide a confidence statement about a service’s long-run tail latency or error probability.

The example supplies synthetic records with 100 requests, a sample p95 of 180 ms and one failed request. It is assessed at a fixed time, with a maximum evidence age of 86,400 seconds. The inclusive inequalities admit the error rate of exactly 1%. Figure 2 shows the acceptance region conditional on sufficient request count and eligible evidence. Its second marked point stipulates two failures, making the report fail the error-rate condition while leaving the latency value unchanged. This perturbation illustrates the rule without adding a measured observation.

Listing 1 reproduces the claim and argument from the executable example. Its collector computes the metrics, and the evidence declaration applies the three thresholds. The reasoning record selects `structured/1`. Consequently, this example demonstrates executable evidence requirements and dependency assessment with an authored rationale; it does not exercise the typed proposition binding illustrated in Figure 1. The complete source and its collector are included in the repository and covered by the test suite.

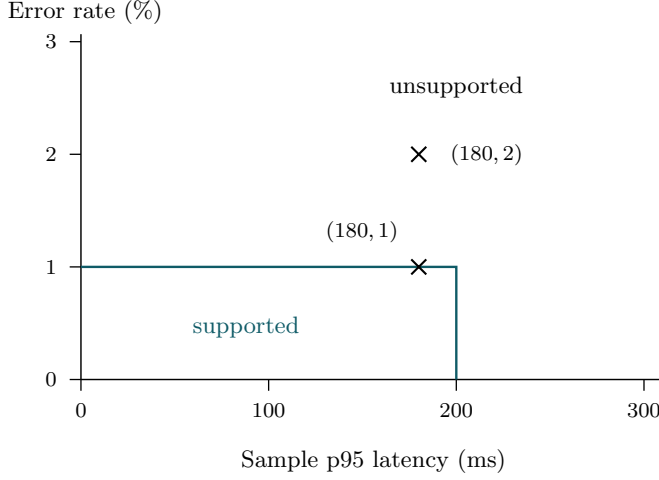
Listing 1: Claim and argument excerpt from `examples/api-load-test/source.eal`. The referenced environment, reasoning and evidence records are declared earlier in that file.

```
claim performance_criteria_met {
  statement "The synthetic report demo-load-001 for orders-api build demo-build-42,
    labelled with 10 concurrent clients, records at least 100 requests with sample p95
    latency at most 200 ms and an error rate at most 1 percent.";
  environment test_run;
}

argument checked_performance {
  conclusion performance_criteria_met;
  reasoning acceptance_criteria;
  evidence load_test;
}
```

Pinning the report digest and comparing command-line and MCP assessments checks whether the same example is interpreted consistently through two interfaces. A digest mismatch, stale report or different build cannot be repaired by a favourable performance number. The operational consequence is to obtain eligible observations before deciding whether the performance requirement holds. This consequence motivates the unavailable category used in the experiment.

Available evidence: $A = 1$



Unavailable: $A = 0$

wrong identity
incomplete records
stale observation
invalid measurements
Performance remains unresolved.

Figure 2: Decision region for the worked API example, conditional on at least 100 requests. With eligible evidence ($A = 1$), the inclusive bounds are sample p95 latency at most 200 ms and error rate at most 1%. The point (180, 1) uses the synthetic example’s values; (180, 2) is a stipulated perturbation. When report validity, identity, completeness, freshness or consistency is unresolved or fails ($A = 0$), performance remains unavailable for this decision. The axes show the declared decision rule; no pilot measurements or fitted response surface are plotted.

6 Historical pilot: materials and method

6.1 Run identity and case construction

The analysed artefact is GitHub Actions run 36163505801, attempt 1, executed on 25 September 2026 using commit 3490076dd78404caa1326d9aae704f964910c4c8. Its manifest identifies plan version 2.0.0 and run schema `eal-api-experiment-run/2`. The supplied ZIP’s SHA-256 agrees with the digest recorded for GitHub artefact 10879620453. Reanalysis uses the original reference and scoring code from that commit. It does not apply the later experiment’s additional comparator or revised outcome fields to the historical answers.

The 40 cases comprise four variations in each of ten designed families: healthy, near-limit, slow, errors, incomplete, wrong identity, stale, corrupt, conflicting and distractor. Each case acquired a real HTTP workload against a controlled loopback service. The harness then constructed a report bundle, including annotated fault injections where required. The harness used the same immutable report bundle for every model and condition assigned to a case. Case identity and byte content are verified during reanalysis. This arrangement controls differences in observations across conditions while retaining a realistic report-selection task. The cases are development examples selected by mechanism, with no probability sampling of production incidents.

The independent reference recomputes metrics from raw request rows and evaluates report selection, identity and validity without calling the EAL collector. A family’s intended label does not determine its reference outcome: measured rows and injected conditions do. The final reference distribution is nine supported, 12 unsupported and 19 unavailable cases. Freshness is evaluated against a shared frozen assessment time with a 300-second age limit. This differs from the worked example’s one-day limit and is preserved in the historical reanalysis.

6.2 Conditions, models and resource limits

The assignment ledger crosses 40 cases, six model snapshots and five conditions, giving 1,200 assignments. The EAL/MCP condition supplies a complete EAL source document and access to the EAL collection and assessment host. The JSON condition supplies the corresponding argument structure as JSON prompt text, with direct collector access. Three prose conditions provide a brief instruction, explicit decision instructions or a review-oriented instruction, also with direct collector access. The historical JSON and prose conditions have no equivalent EAL checker. All conditions receive the same report catalogue and acceptance criteria. Their tools return collector-derived metrics and measurement facts from the same underlying reports; EAL/MCP additionally returns the checked assessment. Prompt lengths and available tool conclusions therefore differ. The comparison concerns these complete systems.

The EAL condition uses `structured/1` with the supplied argument. The pilot therefore tests report selection and communication of a computed assessment; it does not exercise model authoring, typed proposition binding or objection handling. The independent reference uses raw rows, whereas the models report metrics already computed by their collector.

The pinned GPT-4.1 nano, mini and full snapshots are all dated 2025-04-14 and use temperature 0.2. GPT-5.4 nano and mini are dated 2026-03-17; the full snapshot is dated 2026-03-05. The GPT-5.4 configurations request medium reasoning effort. These names identify the archive’s provider snapshots and commercial tiers. The study supplies no parameter counts or controlled manipulation of model capacity.

All six models use the same text-based JSON operation adapter; native provider tool mode is disabled. The EAL condition connects those operations to a real MCP session, while the other conditions call the direct collector. Each trial permits at most six model calls, each with at most 4,096 output tokens and a 120-second timeout. The manifest records a USD 20 per-model admission allowance, a one-second minimum call interval, six workers and assignment seed 20260925. Equal limits do not imply equal prompt tokens or equal computational assistance. In particular, a model requesting an EAL assessment can receive a checked conclusion unavailable to a comparator model.

6.3 Reference decisions and correctness

Let A mean that report validity, identity, completeness, freshness and consistency all evaluate to true. The reference status is unavailable when A is false or unresolved. Given A , it is supported when all three performance predicates hold and unsupported otherwise. This order prevents a missing or invalid observation from being classified as demonstrated poor performance.

The required final answer identifies the report and its scope, gives a status, enumerates failed checks and supplies request count, sample p95 and error rate. The original rubric requires collection, correct report selection, exact scope, correct status, an exact set of failed checks, exact count and latency/error metrics within 0.01 in their reported units. The narrative explanation is retained but is not scored. A failed check is a predicate whose value is explicitly false. An unknown predicate is not included in that set. The version-2 answer schema has no separate `unknown_checks` field.

This distinction is consequential for corrupt reports. Their validity check can be false while other checks and metrics are unknown. Listing every unresolved check as failed violates the rubric even if the answer’s unavailable status is correct. Reporting both the composite score and the underlying discrepancy is necessary to interpret such cases.

The EAL host’s unsupported assessment can arise from a measured failure or from unavailable evidence. The historical adapter therefore supplies both `host_status` and `decision_status`, together with a status mapping. Its presence does not guarantee that a model will select the application-level decision status. Retained host assessments are compared with the independent reference, allowing a disagreement in the final answer to be separated from a disagreement in the

Table 1: Correct assignments out of 40 in each model-condition cell. Failed and unattempted assignments remain in the denominator. Snapshot dates and settings are given in Section 6. “Full” identifies the unqualified model name.

Model	EAL/MCP	JSON	Brief	Explicit	Review
4.1 nano	16/40	6/40	7/40	5/40	3/40
4.1 mini	32/40	21/40	21/40	20/40	21/40
4.1 full	40/40	36/40	33/40	31/40	32/40
5.4 nano	2/40	0/40	0/40	1/40	1/40
5.4 mini	16/40	17/40	16/40	13/40	13/40
5.4 full	0/40	0/40	0/40	0/40	0/40

host’s computation.

6.4 Analysis of incomplete execution

One attempt was assigned to each case-model-condition combination. The archive records 628 completed attempts, 131 failures and 441 assignments not attempted. Of the failures, 128 reached the model-call limit and three returned provider errors stating that the response was incomplete. The runner’s broad provider-error rule stopped subsequent assignments for the affected model. Each GPT-5.4 snapshot encountered such an error; the full snapshot stopped after its first assignment. These stops account for all unattempted assignments.

The primary descriptive score retains the assignment denominator. Write $Y_{ima} = 1$ when case i , model m and condition a completed with a correct answer, and zero otherwise. For each model and condition, the reported score is $\sum_{i=1}^{40} Y_{ima}/40$. This describes the recorded system’s realised yield under the run’s stopping rule. A zero caused by non-execution is not evidence of an incorrect answer that a model would have produced.

For the paired EAL/MCP-JSON contrast, let n_{10} count cases correct only under EAL/MCP and n_{01} those correct only under JSON. The finite-case difference is

$$\Delta_m = \frac{1}{40} \sum_{i=1}^{40} (Y_{imE} - Y_{imJ}) = \frac{n_{10} - n_{01}}{40}. \quad (7)$$

Completed-pair counts accompany this contrast. Conditioning on completion changes the analysed subset and may select easier cases; it cannot repair missing outcomes. No confidence interval, significance test or equivalence conclusion is assigned to these purposively constructed cases. There is one realised model attempt per assignment, shared case structure within each family and no probability sampling scheme for a target population. The observed differences are descriptive development results.

7 Results

7.1 Assignment yield and paired outcomes

Table 1 reports correct assignments across all five conditions. GPT-4.1 full and mini completed all 200 assignments each; their differences therefore do not depend on unattempted cases. Full scored 40/40 in EAL/MCP and 36/40 in JSON, a difference of 10 percentage points. Mini scored 32/40 and 21/40, a difference of 27.5 percentage points. GPT-4.1 nano scored 16/40 and 6/40, but only 25 of those paired assignments completed in both conditions. Its 25-point difference combines answer correctness and completion under the specified limits.

Figure 3 shows the paired counts. For GPT-4.1 mini, EAL/MCP alone was correct on 12 cases and JSON alone on one; the two systems were both incorrect on seven. For full, the

Rows: EAL/MCP correct, incorrect. Columns: JSON correct, incorrect.

4.1 nano	4.1 mini	4.1 full
$\begin{bmatrix} 5 & 11 \\ 1 & 23 \end{bmatrix}$	$\begin{bmatrix} 20 & 12 \\ 1 & 7 \end{bmatrix}$	$\begin{bmatrix} 36 & 4 \\ 0 & 0 \end{bmatrix}$
$\Delta = +25$ percentage points	$\Delta = +27.5$ percentage points	$\Delta = +10$ percentage points
Completed pairs: 25/40	Completed pairs: 40/40	Completed pairs: 40/40
5.4 nano	5.4 mini	5.4 full
$\begin{bmatrix} 0 & 2 \\ 0 & 38 \end{bmatrix}$	$\begin{bmatrix} 9 & 7 \\ 8 & 16 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 \\ 0 & 40 \end{bmatrix}$
$\Delta = +5$ percentage points	$\Delta = -2.5$ percentage points	$\Delta = +0$ percentage points
Completed pairs: 4/40	Completed pairs: 13/40	Completed pairs: 0/40

Each matrix retains all 40 assigned pairs. Off-diagonal counts determine Δ .

Figure 3: Paired EAL/MCP and JSON assignment outcomes. Rows are EAL/MCP correct and incorrect; columns are JSON correct and incorrect. Each matrix sums to all 40 assigned cases. The off-diagonal counts give Δ through Equation 7; completed-pair counts disclose exposure. Here “incorrect” includes failed and unattempted assignments under the declared yield measure. In particular, the 40 in the GPT-5.4 full lower-right cell represents absent successful execution, not 40 observed pairs of wrong answers.

four discordant cases all favoured EAL/MCP under the composite rubric. These counts locate agreement and disagreement more precisely than marginal percentages alone.

The GPT-5.4 records support a narrower account because model-level stopping removed substantial exposure. Mini scored 16/40 in EAL/MCP and 17/40 in JSON, with only 13 completed pairs. Nano scored 2/40 and 0/40, with four completed pairs. Full supplied no completed assignment. Table 2 separates completion, failure and non-execution for each model. These totals preclude interpreting the GPT-5.4 scores as an ordered capability comparison with GPT-4.1.

Table 2: Disposition of the 200 assignments for each model, pooled over five conditions. A failed attempt consumed execution; an unattempted assignment remained after stopping.

Model	Complete	Failed	Not attempted
4.1 nano	123	77	0
4.1 mini	200	0	0
4.1 full	200	0	0
5.4 nano	25	4	171
5.4 mini	80	49	71
5.4 full	0	1	199

Table 3: Recorded token-cost estimates in USD, rounded to four decimal places. Rates are those frozen in the run manifest, not current prices. Zero cells have no recorded model calls.

Model	EAL/MCP	JSON	Brief	Explicit	Review
4.1 nano	0.0211	0.0317	0.0260	0.0301	0.0343
4.1 mini	0.0682	0.0702	0.0762	0.0761	0.0793
4.1 full	0.3500	0.3542	0.3020	0.3063	0.3070
5.4 nano	0.0313	0.0174	0.0308	0.0327	0.0278
5.4 mini	0.4117	0.3256	0.2834	0.3006	0.3207
5.4 full	0.0000	0.0000	0.0000	0.0641	0.0000

7.2 What the correctness differences mean

All four incorrect JSON answers from GPT-4.1 full concerned corrupt-report cases. Their statuses, report selection, scope and metrics were correct; their failed-check sets included unknown checks. Thus the 40 versus 36 composite score supplies evidence about compliance with the answer rubric, while the observed status accuracy was 40/40 in both conditions. A claim that EAL/MCP improved full-model decision-status accuracy would misstate these data.

The eight incorrect EAL/MCP answers from GPT-4.1 mini identify a different problem. Seven had an incorrect status: two incomplete-report cases, four stale-report cases and one wrong-identity case. One further answer used the wrong scope on a healthy case. All 40 mini EAL/MCP answers had correct metrics and failed-check sets. The discrepancy between an accurate computed result and its final status expression makes status communication an interface concern, even when the model successfully obtains the relevant evidence.

Across the run, 141 host checks were recorded, and all agreed with the independent reference according to the archived comparison flags. This is an agreement rate for the checks that were actually invoked and retained; it is not coverage of all 240 EAL assignments. It supports separating the observed communication failures from failures of those recorded host computations. The reference itself remains a program subject to specification and implementation error.

7.3 Consumption and reproducibility

The archive records 2,342 model calls and an estimated total cost of USD 3.97892431 at the manifest’s frozen token rates, with no calls missing cost accounting. Table 3 gives condition totals. They include calls from failed attempts. Missing execution reduces some totals substantially, so lower cost in an incomplete cell does not establish greater efficiency at completing the task.

Each of the three incomplete-response errors occurred on a response reporting 4,096 output tokens, the configured cap. The archive reports 4,073 reasoning tokens for the GPT-5.4 nano response, 4,027 for mini and 38 for full. These observations identify a budget/response interaction requiring investigation. They do not establish the provider’s internal stopping mechanism or predict the result of a larger budget.

The supplied reproduction script verifies the archive-derived file hashes, the 40 case hashes, all 1,200 assignment identities and the two historical scoring-module hashes. It recomputes the independent reference and rubric for all 759 attempted assignments, checks token costs against the frozen rates and regenerates the historical summary. Every recomputed score and the full summary match the archived originals. Reproduction requires no network connection or new model calls. The paper’s numerical tables and paired matrices are generated from that replay, avoiding manual transcription of cell results.

8 Discussion and threats to validity

8.1 Bounded engineering value

The implementation makes several engineering mistakes detectable before a conclusion is communicated: an evidence record can name the wrong subject, use an incompatible unit, fall outside the permitted interval or fail a method-specific input condition. The composed evaluator also exposes unresolved attacks and unavailable premises. These checks provide inspectable reasons for an assessment. Their usefulness depends on engineers choosing quantities, intervals, collectors and methods that represent the intended question.

The pilot adds evidence about the interaction surrounding those checks. On the completely executed GPT-4.1 full and mini strata, EAL/MCP had higher composite correctness than the four supplied alternatives. However, the full-model JSON difference was entirely attributable to the exact failed-check rubric, and mini still miscommunicated seven statuses after using EAL/MCP. These findings support improving the application response schema and its interpretation. They do not establish that an executable host eliminates reasoning errors.

A subsequent study should isolate computational assistance from notation. The later repository protocol adds a plain-validator condition, but no results for that condition appear in the analysed version-2 archive. A comparison with matched checker access, prompt information and output requirements could distinguish the value of argument representation from access to a computed verdict. Model authoring of EAL source would require a separate task and failure taxonomy because this pilot supplies the source in advance.

8.2 Construct and internal validity

The composite outcome combines report acquisition, selection, scope, status, failed-check enumeration and metric reporting. It deliberately demands a complete answer, but its exact-set requirement makes unknown-versus-failed wording consequential. Reporting status accuracy and error components alongside the composite score avoids equating every failed answer with a wrong engineering decision. Explanation quality, calibration, user comprehension and downstream deployment choices were not measured.

The independent reference recomputes results from raw rows, reducing direct implementation reuse with the EAL collector. Both nevertheless implement the same authored specification, and both could embody a mistaken requirement. Case construction, protocol design and interpretation occurred within the project that develops EAL. No independent participant study or external replication is available. Archive integrity and exact score reproduction address data handling; they cannot remove these design dependencies.

The model-level stopping rule makes missingness depend on execution outcomes and assignment order. Treating all assignments as denominators preserves the run's operational yield but penalises conditions scheduled after a model stopped. Restricting analysis to completed trials changes that problem into selection bias. The paper therefore reports both disposition and paired completion and refrains from assigning missing counterfactual answers. A future protocol should distinguish a transient incomplete response from a persistent configuration error, retain the resulting attempts and predefine any retry or continuation rule.

8.3 External and statistical validity

The workload was a local HTTP service with controlled modifications to reports. It supplies real requests and timing observations but does not sample the operating conditions, organisational practices or failure distributions of deployed services. Four variations within each designed family share substantial structure. Forty case identifiers consequently do not imply 40 independent draws from an engineering population.

Provider snapshots, reasoning settings and per-call limits are specific to the archived run. Commercial model tiers are not experimental manipulations of a single capability variable. Cross-family differences combine architecture, training, inference behaviour and interface configuration. Single realised attempts also leave response variability unmeasured. These restrictions motivate descriptive counts and paired differences rather than population confidence intervals or claims of statistical equivalence.

The software account and the pilot concern different commits. The current package’s successful test run does not retroactively validate every historical execution path, and the pilot does not measure later changes. The accompanying materials retain both identities, the original scorer and the artefact digest so that a future study can state exactly which components were changed.

9 Conclusion

EAL/2 provides an executable relation between typed propositions, dated evidence, reasoning methods and defeasible acceptance. Its value is clearest where an engineering assessment must reveal which observation was used, whether it met declared eligibility conditions and how an objection affected the conclusion. The implementation tests and worked example establish bounded software behaviour; they leave the adequacy of the chosen engineering formalisation open to inspection.

Reanalysis of the historical pilot reproduces all recorded scores and preserves incomplete execution. The completely executed GPT-4.1 full and mini strata favour EAL/MCP under the composite rubric, while component analysis narrows what those differences mean. The full-model JSON errors concerned failed-check enumeration, and mini EAL/MCP errors included status communication despite correct metrics. These are specific interface and experimental-design findings. They justify a controlled follow-up with matched validation access and explicit unknown-check reporting, while the present artefact makes the observed results available for review and replication.

Software and data availability

Source and research materials are available in the `emmett08/earl` repository. The implementation description and test run refer to commit `8220e838a8d42922edc0496ff50927c672a1f87d`. The historical experiment refers to commit `3490076dd78404caa1326d9aae704f964910c4c8` and Actions run `36163505801`. The `paper/data` directory preserves all raw case rows and the assignment, answer, outcome and token-accounting fields required for score reproduction. It also contains hashes and provenance for the supplied archive. Full provider response text, replay handles and per-call tool transcripts remain in the original Actions artefact and are not duplicated in the curated scoring dataset. The repository records this archive’s expiry and the need for a durable deposit before journal submission.

Running `make -C paper` regenerates the results, three TikZ figures and manuscript PDF. The reanalysis uses Python’s standard library; figure and manuscript compilation require a LaTeX installation with TikZ, `standalone`, `natbib`, `latexmk` and the packages listed in the source. Author metadata and submission declarations are kept in the accompanying submission checklist for author confirmation before submission.

References

- [1] Phan Minh Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2):321–357, 1995. doi: 10.1016/0004-3702(94)00041-X.

- [2] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799. PMLR, 2023. URL <https://proceedings.mlr.press/v202/gao23f.html>.
- [3] Model Context Protocol maintainers. *Tools*. Model Context Protocol, 2025. URL <https://modelcontextprotocol.io/specification/2025-11-25/server/tools>. Specification revision 2025-11-25.
- [4] Sanjay Modgil and Henry Prakken. The ASPIC+ framework for structured argumentation: A tutorial. *Argument & Computation*, 5(1):31–62, 2014. doi: 10.1080/19462166.2013.869766.
- [5] Object Management Group. *Structured Assurance Case Metamodel (SACM), Version 2.3*, October 2023. URL <https://www.omg.org/spec/SACM/2.3>. Formal document 23-05-08.
- [6] Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 3806–3824. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-emnlp.248. URL <https://aclanthology.org/2023.findings-emnlp.248/>.
- [7] Terence Parr. *Language Implementation Patterns: Create Your Own Domain-Specific and General Programming Languages*. Pragmatic Bookshelf, 2009. ISBN 9781934356456. URL <https://pragprog.com/titles/tpdsl/language-implementation-patterns/>.
- [8] Federico Raspanti, Tanir Ozcelebi, and Mike Holenderski. Grammar-constrained decoding makes large language models better logical parsers. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pages 485–499. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-industry.34. URL <https://aclanthology.org/2025.acl-industry.34/>.